



An efficient dynamic programming algorithm for the generalized LCS problem with multiple substring exclusive constraints



Yingjie Wu^c, Lei Wang^a, Daxin Zhu^b, Xiaodong Wang^{b,c,*}

^a Microsoft AdCenter, Bellevue, WA 98004, USA

^b Faculty of Mathematics & Computer Science, Quanzhou Normal University, China

^c Faculty of Mathematics & Computer Science, Fuzhou University, China

ARTICLE INFO

Article history:

Received 24 March 2013

Received in revised form 4 November 2013

Accepted 24 January 2014

Available online 18 February 2014

Keywords:

Dynamic programming

Algorithm

Generalized LCS problem

Multiple substring exclusion

ABSTRACT

In this paper, we consider a generalized longest common subsequence problem with multiple substring exclusive constraints. For the two input sequences X and Y of lengths n and m , and a set of d constraints $P = \{P_1, \dots, P_d\}$ of total length r , the problem is to find a common subsequence Z of X and Y excluding each of constraint string in P as a substring and the length of Z is maximized. The problem was declared to be NP-hard [7], but we finally found that this is not true. A new dynamic programming solution for this problem is presented in this paper. The correctness of the new algorithm is proved. The time complexity of our algorithm is $O(nmr)$.

© 2014 Elsevier B.V. All rights reserved.

1. Introduction

In this paper, we consider a generalized longest common subsequence problem with multiple substring exclusive constraints. The longest common subsequence (LCS) problem is a well-known measurement for computing the similarity of two strings, and it is crucial in various applications. In this problem, we are interested in a longest sequence which is a subsequence of both sequences. The problem is well studied and is used in many applications, like DNA and protein analysis, text information retrieval, file comparing, music information retrieval, or spelling correction.

The most referred algorithm, proposed by Wagner and Fischer [29], solves the LCS problem by using a dynamic programming algorithm in quadratic time. Other advanced algorithms were proposed in the past decades [3,2,4,16,17,19,21].

If the number of input sequences is not fixed, the problem to find the LCS of multiple sequences has been proved to be NP-hard [23]. Some approximate and heuristic algorithms were proposed for these problems [6,25].

There are also a lot of generalizations of this similarity measure. One of the recent variants of the LCS problem, the constrained longest common subsequence (CLCS) which was first addressed by Tsai [27], has received much attention. It generalizes the LCS measure by introduction of a third sequence, which allows to extort that the obtained CLCS has some special properties [26]. For two given input sequences X and Y of lengths m and n , respectively, and a constrained sequence P of length r , the CLCS problem is to find the common subsequences Z of X and Y such that P is a subsequence of Z and the length of Z is the maximum.

* Corresponding author at: Faculty of Mathematics & Computer Science, Quanzhou Normal University, China.

E-mail address: wangxiaodong@qztc.edu.cn (X. Wang).

Table 1
The GC-LCS problems.

Problem	Input	Output
SEQ-IC-LCS	X, Y , and P	The longest common subsequence of X and Y including P as a subsequence
STR-IC-LCS	X, Y , and P	The longest common subsequence of X and Y including P as a substring
SEQ-EC-LCS	X, Y , and P	The longest common subsequence of X and Y excluding P as a subsequence
STR-EC-LCS	X, Y , and P	The longest common subsequence of X and Y excluding P as a substring

Table 2
The Multiple-GC-LCS problems.

Problem	Input	Output
M-SEQ-IC-LCS	X, Y , and a set of constraints $P = \{P_1, \dots, P_d\}$	The longest common subsequence of X and Y including each of constraint $P_i \in P$ as a subsequence
M-STR-IC-LCS	X, Y , and a set of constraints $P = \{P_1, \dots, P_d\}$	The longest common subsequence of X and Y including each of constraint $P_i \in P$ as a substring
M-SEQ-EC-LCS	X, Y , and a set of constraints $P = \{P_1, \dots, P_d\}$	The longest common subsequence of X and Y excluding each of constraint $P_i \in P$ as a subsequence
M-STR-EC-LCS	X, Y , and a set of constraints $P = \{P_1, \dots, P_d\}$	The longest common subsequence of X and Y excluding each of constraint $P_i \in P$ as a substring

The most referred algorithms were proposed independently [5,8], which solve the CLCS problem in $O(mnr)$ time and space by using dynamic programming algorithms. Some improved algorithms have also been proposed [11,18]. The LCS and CLCS problems on the indeterminate strings were discussed in [20]. Moreover, the problem was extended to the one with weighted constraints, a more generalized problem [24].

Recently, a new variant of the CLCS problem, the restricted LCS problem, was proposed [13], which excludes the given constraint as a subsequence of the answer. The restricted LCS problem becomes NP-hard when the number of constraints is not fixed.

Some more generalized forms of the CLCS problem, the generalized constrained longest common subsequence (GC-LCS) problems, were addressed independently by Chen and Chao [7]. For the two input sequences X and Y of lengths n and m , respectively, and a constraint string P of length r , the GC-LCS problem is a set of four problems which are to find the LCS of X and Y including/excluding P as a subsequence/substring, respectively. The four generalized constrained LCS [7] can be summarized in Table 1.

For the four problems in Table 1, $O(mnr)$ time algorithms were proposed [7]. However, their algorithm for STR-EC-LCS is not correct. In a recent paper, a correct $O(mnr)$ time dynamic programming algorithm was proposed [30]. For all four variants in Table 1, $O(r(m+n) + (m+n)\log(m+n))$ time algorithms were proposed by using the finite automata [12]. Recently, a quadratic algorithm to the STR-IC-LCS problem was proposed [10], and the time complexity of [12] was pointed out not correct.

The four GC-LCS problems can be generalized further to the cases of multiple constraints. In these generalized cases, the single constrained pattern P will be generalized to a set of d constraints $P = \{P_1, \dots, P_d\}$ of total length r , as shown in Table 2.

The problem M-SEQ-IC-LCS has been proved to be NP-hard in [14]. The problem M-SEQ-EC-LCS has also been proved to be NP-hard in [13,28]. In addition, the problems M-STR-IC-LCS and M-STR-EC-LCS were also declared to be NP-hard in [7], but without a proof. The exponential-time algorithms for solving these two problems were also presented in [7].

We will discuss the problem M-STR-EC-LCS in this paper. The failure functions in the Knuth–Morris–Pratt algorithm [22] for solving the string matching problem have been proved very helpful for solving the STR-EC-LCS problem. It has been found by Aho and Corasick [1] that the failure functions can be generalized to the case of keyword tree to speedup the exact string matching of multiple patterns. This idea can be very helpful in our dynamic programming algorithm. This is the main idea of our new algorithm. A polynomial time algorithm is presented for the M-STR-EC-LCS problem based on this observation and disproves that M-STR-EC-LCS problem is NP-hard.

The organization of the paper is as follows.

In the following 4 sections we describe our presented dynamic programming algorithm for the M-STR-EC-LCS problem.

In Section 2 the preliminary knowledge for presenting our algorithm for the M-STR-EC-LCS problem is discussed. In Section 3 we give a new dynamic programming solution for the M-STR-EC-LCS problem with time complexity $O(mnr)$, where n and m are the lengths of the two given input strings, and r is the total length of d constraint strings. In Section 4 we discuss the issues to implement the algorithm efficiently. Some concluding remarks are given in Section 5.

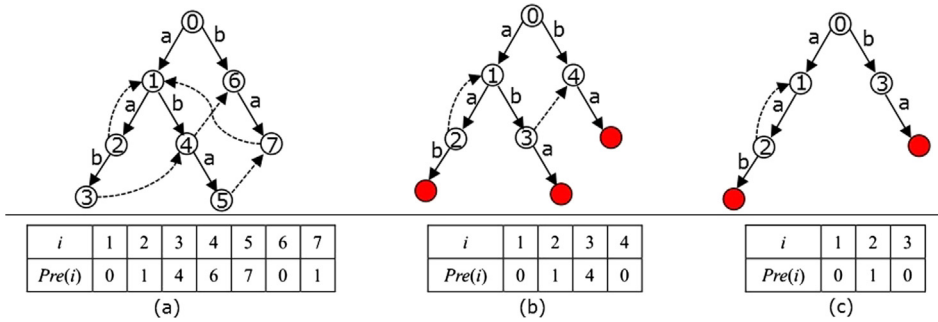


Fig. 1. Keyword trees.

2. Preliminaries

A sequence is a string of characters over an alphabet Σ . A subsequence of a sequence X is obtained by deleting zero or more characters from X (not necessarily contiguous). A substring of a sequence X is a subsequence of successive characters within X .

For a given sequence $X = x_1x_2 \cdots x_n$ of length n , the i th character of X is denoted as $x_i \in \Sigma$ for any $i = 1, \dots, n$. A substring of X from position i to j can be denoted as $X[i : j] = x_ix_{i+1} \cdots x_j$. If $i \neq 1$ or $j \neq n$, then the substring $X[i : j] = x_ix_{i+1} \cdots x_j$ is called a proper substring of X . A substring $X[i : j] = x_ix_{i+1} \cdots x_j$ is called a prefix or a suffix of X if $i = 1$ or $j = n$, respectively.

For the two input sequences $X = x_1x_2 \cdots x_n$ and $Y = y_1y_2 \cdots y_m$ of lengths n and m , respectively, and a set of d constraints $P = \{P_1, \dots, P_d\}$ of total length r , the problem M-STR-EC-LCS is to find an LCS of X and Y excluding each of constraint $P_i \in P$ as a substring.

The most important difference between the problems STR-EC-LCS and M-STR-EC-LCS is the number of constraints. For ease of discussion, we will make the following two assumptions on the constraint set P .

Assumption 1. There are no duplicated strings in the constraint set P .

Assumption 2. No string in the constraint set P is a proper substring of any other string in P .

Keyword tree [1,9,15] is a main data structure in our dynamic programming algorithm to process the constraint set P of the M-STR-EC-LCS problem.

Definition 1. The keyword tree for set P is a rooted directed tree T satisfying 3 conditions: 1. Each edge is labeled with exactly one character; 2. Any two edges out of the same node have distinct labels; and 3. Every string P_i in P maps to some node v of T such that the characters on the path from the root of T to v exactly spell out P_i , and every leaf of T is mapped to some string in P .

For example, Fig. 1(a) shows the keyword tree T for the constraint set $P = \{aab, aba, ba\}$, where $d = 3$, $r = 8$. Clearly, every node in the keyword tree corresponds to a prefix of one of the strings in set P , and every prefix of a string P_i in P maps to a distinct node in the keyword tree T . The keyword tree for set P of total length r of all strings can be easily constructed in $O(r)$ time for a constant alphabet size. Because no two edges out of any node of T are labeled with the same character, the keyword tree T can be used to search for all occurrences in a text X of strings from P .

The failure functions in the Knuth–Morris–Pratt algorithm for solving the string matching problem can be generalized to the case of keyword tree to speedup the exact string matching of multiple patterns as follows.

In order to identify the nodes of T , we assign numbers $0, 1, \dots, t-1$ to all t nodes of T in their pre-order numbering. Then, each node will be assigned an integer i , $0 \leq i < t$, as shown in Fig. 1. For each node numbered i of a keyword tree T , the concatenation of characters on the path from the root to the node i spells out a string denoted as $L(i)$. The string $L(i)$ is also called the label of the node i in the keyword tree T . For any node i of T , define $lp(i)$ to be the length of the longest proper suffix of string $L(i)$ that is a prefix of some string in T .

It can be verified readily that for each node i of T , if A is an $lp(i)$ -length suffix of string $L(i)$, then there must be a unique node $pre(i)$ in T such that $L(pre(i)) = A$. If $lp(i) = 0$ then $pre(i) = 0$ is the root of T .

Definition 2. The ordered pair $(i, pre(i))$ is called a failure link.

The failure link is a direct generalization of the failure functions in the KMP algorithm. For example, in Fig. 1(a), failure links are shown as pointers from every node i to node $pre(i)$ where $lp(i) > 0$. The other failure links point to the root and are not shown.

The failure links of T define actually a failure function pre for the constraint set P .

For example, for the nodes $i = 1, 2, 3, 4, 5, 6, 7$, the corresponding values of failure function are $pre(i) = 0, 1, 4, 6, 7, 0, 1$, as shown in Fig. 1(a).

The failure function pre is used to speedup the search for all occurrences in a text X of strings from P . As stated in [1,9], the failure function pre can be computed in $O(r)$ time.

In the keyword tree application in our dynamic programming algorithm, a function σ will be mentioned frequently. For a string S and a given keyword tree T , if the label $L(i)$ of a node numbered i is also a suffix of S , then the node i is called a suffix node of S in T .

Definition 3. For any string S and a given keyword tree T , the unique suffix node of S in T with maximum depth is denoted as $\sigma(S)$. That is:

$$|L(\sigma(S))| = \max_{0 \leq i < t} \{|L(i)| \mid L(i) \text{ is a suffix of } S\} \quad (1)$$

where t is the number of nodes in T .

For example, if $S = aabaaabb$, then in the keyword tree T of Fig. 1(a), the node 6 is the only suffix node of S in T , therefore $\sigma(S) = 6$.

In our keyword tree application, we are only interested in the nonleaf nodes of the tree. So, we can renumber the nodes of the tree only for nonleaf nodes, omitting the leaf nodes of the tree, as shown in Fig. 1(b). After renumbering, the failure function of the tree will also be changed accordingly.

If a string P_i in the constraint set P is a proper substring of another string P_j in P , then an LCS of X and Y excluding P_i must also exclude P_j . For this reason, the constraint string P_j can be removed from constraint set P without changing the solution of the problem. For example, the string ba is a proper substring of the string aba in the keyword tree of Fig. 1(b). Therefore, the string aba can be removed from the keyword tree, as shown in Fig. 1(c). We will show shortly how to remove these redundant strings from constraint set P in $O(r)$ time. In the following sections, discussions are based on Assumptions 1 and 2 on the constraint set P . The number of nonleaf nodes of the keyword tree for the constraint set P is denoted as s . In the worst case $s = r - d$. The root of the keyword tree is numbered 0, and the other nonleaf nodes are numbered $1, 2, \dots, s - 1$ in their pre-order numbering. For example, in Fig. 1(c), there are $s = 4$ nonleaf nodes in T . The labels for the four nonleaf nodes are $L(0) = \emptyset$, $L(1) = a$, $L(2) = aa$ and $L(3) = b$ respectively.

The symbol $\oplus$ is also used to denote the string concatenation. For example, if $S_1 = aaa$ and $S_2 = bbb$, then it is readily seen that $S_1 \oplus S_2 = aaabbb$.

3. Our main result: a dynamic programming algorithm

In the following discussions, we will call ‘a sequence excluding each of constraint string in P as a substring’ a sequence excluding P for short.

Definition 4. Let $Z(i, j, k)$ denote the set of all LCSs of $X[1 : i]$ and $Y[1 : j]$ excluding P and $\sigma(z) = k$ for each $z \in Z(i, j, k)$, where $1 \leq i \leq n$, $1 \leq j \leq m$, and $0 \leq k < s$. The length of an LCS in $Z(i, j, k)$ is denoted as $f(i, j, k)$.

If we can compute $f(i, j, k)$ for any $1 \leq i \leq n$, $1 \leq j \leq m$, and $0 \leq k < s$ efficiently, then the length of an LCS of X and Y excluding P must be $\max_{0 \leq k < s} \{f(n, m, k)\}$.

By using the keyword tree data structure described in the last section, we can give a recursive formula for computing $f(i, j, k)$ by the following theorem.

Theorem 1. For the two input sequences $X = x_1x_2 \dots x_n$ and $Y = y_1y_2 \dots y_m$ of lengths n and m , respectively, and a set of d constraints $P = \{P_1, \dots, P_d\}$ of total length r , let $Z(i, j, k)$ and $f(i, j, k)$ be defined as in Definition 4. Suppose a keyword tree T for the constraint set P has been built, and the s nonleaf nodes of T are numbered in their pre-order numbering. The label of the node numbered k ($0 \leq k < s$) is denoted as $L(k)$. Then, for any $1 \leq i \leq n$, $1 \leq j \leq m$, and $0 \leq k < s$, $f(i, j, k)$ can be computed by the following recursive formula (2).

$$f(i, j, k) = \begin{cases} \max\{f(i-1, j, k), f(i, j-1, k)\} & \text{if } x_i \neq y_j, \\ \max\{f(i-1, j-1, k), 1 + \max_{0 \leq q < s} \{f(i-1, j-1, q) \mid \sigma(L(q) \oplus x_i) = k\}\} & \text{if } x_i = y_j. \end{cases} \quad (2)$$

The boundary conditions of this recursive formula are $f(i, 0, k) = f(0, j, k) = 0$ for any $0 \leq i \leq n$, $0 \leq j \leq m$, and $0 \leq k \leq s$.

Proof. For any $1 \leq i \leq n$, $1 \leq j \leq m$, and $0 \leq k < s$, suppose $f(i, j, k) = t$ and $z = z_1 \dots z_t \in Z(i, j, k)$.

First of all, we notice that for each pair (i', j') , $1 \leq i' \leq n$, $1 \leq j' \leq m$, such that $i' \leq i$ and $j' \leq j$, we have $f(i', j', k) \leq f(i, j, k)$, since a common subsequence z of $X[1 : i']$ and $Y[1 : j']$ excluding P and $\sigma(z) = k$ is also a common subsequence of $X[1 : i]$ and $Y[1 : j]$ excluding P and $\sigma(z) = k$.

(1) In the case of $x_i \neq y_j$, we have $x_i \neq z_t$ or $y_j \neq z_t$.

(1.1) If $x_i \neq z_t$, then $z = z_1 \cdots z_t$ is a common subsequence of $X[1 : i - 1]$ and $Y[1 : j]$ excluding P and $\sigma(z_1 \cdots z_t) = k$, and so $f(i - 1, j, k) \geq t$. On the other hand, $f(i - 1, j, k) \leq f(i, j, k) = t$. Therefore, in this case we have $f(i, j, k) = f(i - 1, j, k)$.

(1.2) If $y_j \neq z_t$, then we can prove similarly that in this case, $f(i, j, k) = f(i, j - 1, k)$.

Combining the two subcases we conclude that in the case of $x_i \neq y_j$, we have

$$f(i, j, k) = \max\{f(i - 1, j, k), f(i, j - 1, k)\}.$$

(2) In the case of $x_i = y_j$, there are also two cases to be distinguished.

(2.1) If $x_i = y_j \neq z_t$, then $z = z_1 \cdots z_t$ is also a common subsequence of $X[1 : i - 1]$ and $Y[1 : j - 1]$ excluding P and $\sigma(z_1 \cdots z_t) = k$, and so $f(i - 1, j - 1, k) \geq t$. On the other hand, $f(i - 1, j - 1, k) \leq f(i, j, k) = t$. Therefore, in this case we have $f(i, j, k) = f(i - 1, j - 1, k)$.

(2.2) If $x_i = y_j = z_t$, then $f(i, j, k) = t > 0$ and $z = z_1 \cdots z_t$ is an LCS of $X[1 : i]$ and $Y[1 : j]$ excluding P and $\sigma(z_1 \cdots z_t) = k$, and thus $z_1 \cdots z_{t-1}$ is a common subsequence of $X[1 : i - 1]$ and $Y[1 : j - 1]$ excluding P .

Let $\sigma(z_1, \dots, z_{t-1}) = q$ and $f(i - 1, j - 1, q) = h$. Then $z_1 \cdots z_{t-1}$ is a common subsequence of $X[1 : i - 1]$ and $Y[1 : j - 1]$ excluding P and $\sigma(z_1 \cdots z_{t-1}) = q$. Therefore, we have

$$f(i - 1, j - 1, q) = h \geq t - 1. \quad (3)$$

Let $v = v_1 \cdots v_h \in Z(i - 1, j - 1, q)$ is an LCS of $X[1 : i - 1]$ and $Y[1 : j - 1]$ excluding P and $\sigma(v_1 \cdots v_h) = q$. Then $\sigma((v_1 \cdots v_h) \oplus x_i) = \sigma(L(q) \oplus x_i) = k$, and thus $(v_1 \cdots v_h) \oplus x_i$ is a common subsequence of $X[1 : i]$ and $Y[1 : j]$ excluding P and $\sigma((v_1 \cdots v_h) \oplus x_i) = k$.

Therefore,

$$f(i, j, k) = t \geq h + 1. \quad (4)$$

Combining (3) and (4) we have $h = t - 1$. Therefore, $z_1 \cdots z_{t-1}$ is an LCS of $X[1 : i - 1]$ and $Y[1 : j - 1]$ excluding P and $\sigma(z_1 \cdots z_{t-1}) = q$.

In other words,

$$f(i, j, k) \leq 1 + \max_{0 \leq q < s} \{f(i - 1, j - 1, q) \mid \sigma(L(q) \oplus x_i) = k\} \quad (5)$$

On the other hand, for any $0 \leq q < s$, if $f(i - 1, j - 1, q) = h$ and $\sigma(L(q) \oplus x_i) = k$, then for any $v = v_1 \cdots v_h \in Z(i - 1, j - 1, q)$, $v \oplus x_i$ is a common subsequence of $X[1 : i]$ and $Y[1 : j]$ and $\sigma(v \oplus x_i) = k$. Since v excludes P and $\sigma(v \oplus x_i) = k < s$, $v \oplus x_i$ is a common subsequence of $X[1 : i]$ and $Y[1 : j]$ excluding P . Furthermore, $v \oplus x_i$ is a common subsequence of $X[1 : i]$ and $Y[1 : j]$ excluding P and $\sigma(v \oplus x_i) = k$. Therefore, $f(i, j, k) = t \geq 1 + h = 1 + f(i - 1, j - 1, q)$, and so we conclude that,

$$f(i, j, k) \geq 1 + \max_{0 \leq q < s} \{f(i - 1, j - 1, q) \mid \sigma(L(q) \oplus x_i) = k\} \quad (6)$$

Combining (5) and (6) we have, in this case,

$$f(i, j, k) = 1 + \max_{0 \leq q < s} \{f(i - 1, j - 1, q) \mid \sigma(L(q) \oplus x_i) = k\} \quad (7)$$

Combining the two subcases in the case of $x_i = y_j$, we conclude that the recursive formula (2) is correct for the case $x_i = y_j$.

The proof is complete. $\square$

4. The implementation of the algorithm

According to Theorem 1, our algorithm for computing $f(i, j, k)$ is a standard 3-dimensional dynamic programming algorithm. By the recursive formula (2), the dynamic programming algorithm for computing $f(i, j, k)$ can be implemented as the following Algorithm 1.

In Algorithm 1, s is the number of nonleaf nodes of the keyword tree T for set P . The root of the keyword tree is numbered 0, and the other nonleaf nodes are numbered $1, 2, \dots, s - 1$ in their pre-order numbering. $L(t)$ is the label of node numbered t in the keyword tree T .

To implement our algorithm efficiently, the most important thing is to compute $\sigma(L(k) \oplus x_i)$ for each $0 \leq k < s$ and x_i , $1 \leq i \leq n$, in line 11 efficiently.

It is obvious that $\sigma(L(k) \oplus x_i) = g$ if there is an edge (k, g) out of the node k labeled x_i . It will be more complex to compute $\sigma(L(k) \oplus x_i)$ if there is no edge out of the node k labeled x_i . In this case the matched node label has to be changed to the longest proper suffix of $L(k)$ that is a prefix of some string in T and such that the corresponding node h has an out edge (h, g) labeled x_i . Therefore, in this case, $\sigma(L(k) \oplus x_i) = g$.

This computation is very similar to the search algorithm in the keyword tree T for the multiple string matching problem [1,9].

Algorithm 1 M-STR-EC-LCS.

Input: Strings $X = x_1 \cdots x_n$, $Y = y_1 \cdots y_m$ of lengths n and m , respectively, and a set of d constraints $P = \{P_1, \dots, P_d\}$ of total length r

Output: The length of an LCS of X and Y excluding P

```

1: Build a keyword tree  $T$  for  $P$ 
2: for all  $i, j, k$ ,  $0 \leq i \leq n$ ,  $0 \leq j \leq m$ , and  $0 \leq k \leq s$  do
3:    $f(i, 0, k) \leftarrow 0$ ,  $f(0, j, k) \leftarrow 0$  {boundary condition}
4: end for
5: for  $i = 1$  to  $n$  do
6:   for  $j = 1$  to  $m$  do
7:     for  $k = 0$  to  $s$  do
8:       if  $x_i \neq y_j$  then
9:          $f(i, j, k) \leftarrow \max\{f(i-1, j, k), f(i, j-1, k)\}$ 
10:      else
11:         $u \leftarrow \max_{0 \leq t < s} \{f(i-1, j-1, t) \mid \sigma(L(t) \oplus x_i) = k\}$ 
12:         $f(i, j, k) \leftarrow \max\{f(i-1, j-1, k), 1+u\}$ 
13:      end if
14:    end for
15:  end for
16: end for
17: return  $\max_{0 \leq t < s} \{f(n, m, t)\}$ 

```

With pre-computed prefix function pre , the function $\sigma(L(k) \oplus ch)$ for each character $ch \in \Sigma$ and $1 \leq k \leq s$ can be computed in $O(1)$ time. Then, we can compute an index t^* such that

$$f(i-1, j-1, t^*) = \max_{0 \leq t < s} \{f(i-1, j-1, t) \mid \sigma(L(t) \oplus x_i) = k\}$$

in line 11 of Algorithm 1 by an algorithm $\max \sigma(i, j, k)$.

Algorithm 2 $\max \sigma(i, j, k)$.

Input: Integers i, j, k

Output: An index t^* such that $f(i-1, j-1, t^*) = \max_{0 \leq t < s} \{f(i-1, j-1, t) \mid \sigma(L(t) \oplus x_i) = k\}$

```

1:  $t_{mp} \leftarrow -1$ ,  $t^* \leftarrow -1$ 
2: for  $t = 0$  to  $s-1$  do
3:   if  $\sigma(t, x_i) = k$  and  $f(i-1, j-1, t) > t_{mp}$  then
4:      $t_{mp} \leftarrow f(i-1, j-1, t)$ ,  $t^* \leftarrow t$ 
5:   end if
6: end for
7: return  $t^*$ 

```

Then the value of u in line 11 of Algorithm 1 must be

$$u = f(i-1, j-1, t^*) = f(i-1, j-1, \max \sigma(i, j, k)).$$

We can improve the efficiency of Algorithms 1 further in following two points.

First, we can pre-compute a table λ of the function $\sigma(L(k) \oplus ch)$ for each character $ch \in \Sigma$ and $1 \leq k \leq s$ to speed up the computation of $\max \sigma(i, j, k)$. When we pre-compute the prefix function pre , for every edge (k, g) labeled with character ch , the value of $\lambda(k, ch)$ can be assigned directly to g . These computations can be done when we build the keyword tree T for the constraint set P , or we can traverse the keyword tree T in a pre-order manner as follows.

Algorithm 3 $PreOrder(t)$.

```

1: if  $t \leq s$  then
2:   for all  $g$  such that  $(t, a) = g$  is an edge out  $t$  do
3:      $\lambda(t, a) \leftarrow g$ 
4:      $PreOrder(g)$ 
5:   end for
6: end if

```

The other values of the table λ can be computed by using the prefix function pre . The whole pre-computation of the table λ can be described in the following algorithm.

The time cost of computing all values $\lambda(k, ch)$ of the table for each character $ch \in \Sigma$ and $1 \leq k \leq s$ by above preprocessing algorithm is obviously $O(s|\Sigma|)$. By using this pre-computed table λ , the value of function $\sigma(L(k) \oplus ch)$ (which is equal to $\lambda(L(k), ch)$) for each character $ch \in \Sigma$ and $1 \leq k < s$ can be computed readily in $O(1)$ time.

Second, the computation of function $\max \sigma(i, j, k)$ is very time consuming and many repeated computations are overlapped in the whole **for** loop of Algorithm 1. We can use a forward-dynamic programming method to each entry of $f(i, j, k)$ in the **for** loop on variable k of Algorithm 1 and finally reduce the time costs of the whole algorithm. The modified algorithm can be described as follows.

Algorithm 4 Pre-computation of λ .

```

1: for all  $1 \leq k \leq s$  and  $ch \in \Sigma$  do
2:    $\lambda(k, ch) \leftarrow -1$ 
3: end for
4: PreOrder(0)
5: for  $k = 1$  to  $s$  do
6:   for all  $ch \in \Sigma$  do
7:     if  $\lambda(k, ch) < 0$  then
8:        $\lambda(k, ch) \leftarrow \lambda(pre(k), ch)$ 
9:     end if
10:   end for
11: end for

```

Algorithm 5 M-STR-EC-LCS.

Input: Strings $X = x_1 \dots x_n$, $Y = y_1 \dots y_m$ of lengths n and m , respectively, and a set of d constraints $P = \{P_1, \dots, P_d\}$ of total length r

Output: The length of an LCS of X and Y excluding P

```

1: Build a keyword tree  $T$  for  $P$ 
2: for all  $i, j, k$ ,  $0 \leq i \leq n$ ,  $0 \leq j \leq m$ , and  $0 \leq k \leq s$  do
3:    $f(i, 0, k) \leftarrow 0$ ,  $f(0, j, k) \leftarrow 0$  {boundary condition}
4: end for
5: for  $i = 1$  to  $n$  do
6:   for  $j = 1$  to  $m$  do
7:     for  $k = 0$  to  $s$  do
8:        $f(i, j, k) \leftarrow \max\{f(i-1, j, k), f(i, j-1, k)\}$ 
9:     end for
10:    if  $x_i = y_j$  then
11:      for  $k = 0$  to  $s$  do
12:         $t \leftarrow \lambda(k, x_i)$ 
13:         $f(i, j, t) \leftarrow \max\{f(i-1, j-1, t), 1 + f(i-1, j-1, k)\}$ 
14:      end for
15:    end if
16:  end for
17: end for
18: return  $\max_{0 \leq t \leq s} \{f(n, m, t)\}$ 

```

Since $\lambda(k, x_i)$ can be computed in $O(1)$ time for each x_i , $1 \leq i \leq n$ and any $0 \leq k < s$, the loop body of our improved Algorithm 5 requires only $O(1)$ time. Therefore, our dynamic programming algorithm for computing the length of an LCS of X and Y excluding P requires $O(nmr)$ time and $O(r|\Sigma|)$ preprocessing time.

Until now we have assumed that our algorithm is implemented under Assumption 1 and Assumption 2 on the constraint set P . We now describe how to relax the two assumptions.

If Assumption 1 is violated, then there must be some duplicated strings in the constraint set P . In this case, we can first sort the strings in the constraint set P , then duplicated strings can be removed from P easily and then Assumption 1 on the constraint set P is satisfied. It is clear that removed strings will not change the solution of the problem.

For Assumption 2, we first notice that a string A in the constraint set P is a proper substring of string B in P , if and only if in the keyword tree T of P , there is a directed path of failure links from a node v on the path from the root to the leaf node corresponding to string B to the leaf node corresponding to string A [1,9]. For example, in Fig. 1(a), there is a directed path of failure links from node 5 to node 7 and thus we know the string ba corresponding to node 7 is a proper substring of string aba corresponding to node 5.

With this fact, if Assumption 2 is violated, we can remove all super-strings from the constraint set P as follows. We first build a keyword tree T for the constraint set P , then mark all nodes passed by a directed path of failure links to a leaf node in T by using a depth first traversal of T . All the strings corresponding to the marked leaf node can then be removed from P . Assumption 2 is now satisfied on the new constraint set and the keyword tree T for the new constraint set is then rebuilt. It is not difficult to do this preprocessing in $O(r)$ time. It is clear that the removed super-strings will not change the solution of the problem.

If we want to compute the longest common subsequence of X and Y excluding P , but not just its length, we can also present a simple recursive back tracing algorithm for this purpose as the following Algorithm 6.

In the end of our new algorithm, we will find an index t such that $f(n, m, t)$ gives the length of an LCS of X and Y excluding P . Then, a function call $back(n, m, t)$ will produce the answer LCS accordingly.

Since the cost of the algorithm $\max_{\sigma} \sigma(i, j, k)$ is $O(r)$ in the worst case, the time complexity of the algorithm $back(i, j, k)$ is $O(r \max(n, m))$.

Finally we summarize our results in the following theorem.

Theorem 2. Algorithm 5 solves the M-STR-EC-LCS problem correctly in $O(nmr)$ time and $O(nmr)$ space, with preprocessing time $O(r|\Sigma|)$.

Algorithm 6 *back*(*i*, *j*, *k*).**Comments:** A recursive back tracing algorithm to construct the answer LCS

```

1: if i = 0 or j = 0 then
2:   return
3: end if
4: if  $x_i = y_j$  then
5:   if  $f(i, j, k) = f(i - 1, j - 1, k)$  then
6:     back(i - 1, j - 1, k)
7:   else
8:     back(i - 1, j - 1, max  $\sigma(i, j, k)$ )
9:   print  $x_i$ 
10: end if
11: else if  $f(i - 1, j, k) > f(i, j - 1, k)$  then
12:   back(i - 1, j, k)
13: else
14:   back(i, j - 1, k)
15: end if

```

5. Concluding remarks

We have suggested a new dynamic programming solution for the M-STR-EC-LCS problem. The M-STR-IC-LCS problem is another interesting generalized constrained longest common subsequence (GC-LCS) which is very similar to the M-STR-EC-LCS problem. The M-STR-IC-LCS problem is to find an LCS of two main sequences, in which a set of constraint strings must be included as its substrings. It is not clear that whether the same technique of this paper can be applied to this problem to achieve an efficient algorithm. We will investigate the problem further.

References

- [1] A.V. Aho, M.J. Corasick, Efficient string matching: an aid to bibliographic search, *Commun. ACM* 18 (6) (1975) 333–340.
- [2] H.Y. Ann, C.B. Yang, Y.H. Peng, B.C. Liaw, Efficient algorithms for the block edit problems, *Inf. Comput.* 208 (3) (2010) 221–229.
- [3] H.Y. Ann, C.B. Yang, C.T. Tseng, C.Y. Hor, A fast and simple algorithm for computing the longest common subsequence of run-length encoded strings, *Inf. Process. Lett.* 108 (11) (2008) 360–364.
- [4] A. Apostolico, C. Guerra, The longest common subsequences problem revisited, *Algorithmica* 2 (1) (1987) 315–336.
- [5] A.N. Arslan, O. Egencioglu, Algorithms for the constrained longest common subsequence problems, *Int. J. Found. Comput. Sci.* 16 (6) (2005) 1099–1109.
- [6] C. Blum, M.J. Blesa, M. López-Ibáñez, Beam search for the longest common subsequence problem, *Comput. Oper. Res.* 36 (12) (2009) 3178–3186.
- [7] Y.C. Chen, K.M. Chao, On the generalized constrained longest common subsequence problems, *J. Comb. Optim.* 21 (3) (2011) 383–392.
- [8] F.Y.L. Chin, A.D. Santis, A.L. Ferrara, N.L. Ho, S.K. Kim, A simple algorithm for the constrained sequence problems, *Inf. Process. Lett.* 90 (4) (2004) 175–179.
- [9] M. Crochemore, C. Hancart, T. Lecroq, *Algorithms on Strings*, Cambridge University Press, Cambridge, UK, 2007.
- [10] S. Deorowicz, Quadratic-time algorithm for a string constrained LCS problem, *Inf. Process. Lett.* 112 (11) (2012) 423–426.
- [11] S. Deorowicz, J. Ostoj, Constrained longest common subsequence computing algorithms in practice, *Comput. Inform.* 29 (3) (2010) 427–445.
- [12] E. Farhana, J. Ferdous, T. Moosa, M.S. Rahman, Finite automata based algorithms for the generalized constrained longest common subsequence problems, in: *Proceedings of the 17th International Conference on String Processing and Information Retrieval, SPIRE'10*, Los Cabos, Mexico, 2010, pp. 243–249.
- [13] Z. Gotthilf, D. Hermelin, G.M. Landau, M. Lewenstein, LCS. Restricted, in: *Proceedings of the 17th International Conference on String Processing and Information Retrieval, SPIRE'10*, Los Cabos, Mexico, 2010, pp. 250–257.
- [14] Z. Gotthilf, D. Hermelin, M. Lewenstein, Constrained LCS: hardness and approximation, in: *Proceedings of the 19th Annual Symposium on Combinatorial Pattern Matching, CPM'08*, Pisa, Italy, 2008, pp. 255–262.
- [15] D. Gusfield, *Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology*, Cambridge University Press, Cambridge, UK, 1997.
- [16] D.S. Hirschberg, Algorithms for the longest common subsequence problem, *J. ACM* 24 (4) (1977) 664–675.
- [17] J.W. Hunt, T.G. Szymanski, A fast algorithm for computing longest common subsequences, *Commun. ACM* 20 (5) (1977) 350–353.
- [18] C.S. Iliopoulos, M.S. Rahman, New efficient algorithms for the LCS and constrained LCS problems, *Inf. Process. Lett.* 106 (1) (2008) 13–18.
- [19] C.S. Iliopoulos, M.S. Rahman, A new efficient algorithm for computing the longest common subsequence, *Theor. Comput. Sci.* 45 (2) (2009) 355–371.
- [20] C.S. Iliopoulos, M.S. Rahman, W. Rytter, Algorithms for two versions of LCS problem for indeterminate strings, *J. Comb. Math. Comb. Comput.* 71 (2009) 155–172.
- [21] C.S. Iliopoulos, M.S. Rahman, M. Voráček, L. Vagner, Finite automata based algorithms on subsequences and supersequences of degenerate strings, *J. Discrete Algorithms* 8 (2) (2010) 117–130.
- [22] D.E. Knuth, J.H. Morris Jr, V. Pratt, Fast pattern matching in strings, *SIAM J. Comput.* 6 (2) (1977) 323–350.
- [23] D. Maier, The complexity of some problems on subsequences and supersequences, *J. ACM* 25 (1978) 322–336.
- [24] Y.H. Peng, C.B. Yang, K.S. Huang, K.T. Tseng, An algorithm and applications to sequence alignment with weighted constraints, *Int. J. Found. Comput. Sci.* 21 (1) (2010) 51–59.
- [25] S.J. Shyu, C.Y. Tsai, Finding the longest common subsequence for multiple biological sequences by ant colony optimization, *Comput. Oper. Res.* 36 (1) (2009) 73–91.
- [26] C.Y. Tang, C.L. Lu, Constrained multiple sequence alignment tool development and its application to RNase family alignment, *J. Bioinform. Comput. Biol.* 1 (2003) 267–287.
- [27] Y.T. Tsai, The constrained longest common subsequence problem, *Inf. Process. Lett.* 88 (4) (2003) 173–176.
- [28] C.T. Tseng, C.B. Yang, H.Y. Ann, Efficient algorithms for the longest common subsequence problem with sequential substring constraints, *J. Complex.* 29 (2013) 44–52.
- [29] R. Wagner, M. Fischer, The string-to-string correction problem, *J. ACM* 21 (1) (1974) 168–173.
- [30] L. Wang, X. Wang, Y. Wu, D. Zhu, A dynamic programming solution to a generalized LCS problem, *Inf. Process. Lett.* 113 (1) (2013) 723–728.